

Unit-8: Designing GUI Applications using JavaFX

Unit-8: Topics & Subtopics Table

Main Topic	Subtopics	What You Learn
1. Introduction to JavaFX	What is JavaFX	GUI framework for Java
	Difference: AWT vs Swing vs JavaFX	Evolution of GUI
	Features of JavaFX	Modern UI, CSS, animation
2. Basic Structure of JavaFX Application	Application class	Entry point
	start() method	UI initialization
	Stage	Main window
	Scene	Content container
	Node	UI elements
3. JavaFX Architecture (Core Understanding)	Stage → Scene → Node hierarchy	UI structure
	Scene graph	Tree structure of UI
	Layout vs Controls	UI organization
4. Layout Panes	Pane (basic container)	Free positioning
	VBox	Vertical layout
	HBox	Horizontal layout
	GridPane	Table-like layout
	BorderPane	Top, bottom, left, right
	StackPane	Overlapping nodes
5. UI Components (Controls)	Label	Display text
	Button	Click action
	TextField	Input field
	TextArea	Multi-line input
	CheckBox	Selection
	RadioButton	Single choice
	ComboBox	Dropdown
6. Text, Font, Color	Text class	Display styled text
	Font class	Font style/size
	Color class	Text/UI color
	Styling UI	Basic design
7. Event Handling	Event concept	User interaction
	ActionEvent	Button click
	Event handler	Handling actions
	Lambda expressions	Clean event code

8. JavaFX Properties (Basic)	Observable properties	Dynamic UI updates
	Binding concept	Auto UI updates
9. Animation (Basic idea)	Timeline	Frame-based animation
	Transition classes	Movement
10. Real-World GUI Applications	Forms	Registration UI
	Dashboards	Data display
	Interactive apps	Games, tools

Unit-8: Introduction to JavaFX

This section builds **strong foundation** of GUI development and how JavaFX fits into Java ecosystem.

1. What is JavaFX (GUI Framework for Java)

Definition: JavaFX is a modern GUI framework used to build desktop applications with graphical interfaces in Java.

Core Idea: JavaFX → Create windows, buttons, forms, animations

Deep Understanding

Earlier, Java programs were: Console-based → text input/output

JavaFX allows: Graphical UI → buttons, images, layouts, animations

Example

Instead of:

Enter name: Dhruv

You build:

Textbox + Button + Label

Key Characteristics

Feature	Explanation
GUI-based	Visual applications
Event-driven	Reacts to user actions
Rich UI	Styles, animations
Cross-platform	Runs anywhere Java runs

Simple Example (Basic JavaFX Structure)

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
```

```
public class Test extends Application {
    public void start(Stage stage) {
        Button btn=new Button("Click Me");
        StackPane root=new StackPane(btn);
        Scene scene=new Scene(root,300,200);
        stage.setTitle("JavaFX App");
        stage.setScene(scene);
        stage.show();
    }
}
```

```
public static void main(String[] args) {  
    launch(args);  
}  
}
```

What Happens

Stage → window

Scene → content

Button → UI element

2. Evolution of GUI (AWT → Swing → JavaFX)

Java GUI evolved in 3 stages:

1. AWT (Abstract Window Toolkit)

Concept: First GUI framework in Java.

Features

- Uses OS native components
- Limited UI
- Platform dependent

Problems

Heavyweight → slow

Limited customization

2. Swing

Concept: Improved version of AWT.

Features

- Pure Java (lightweight)
- More components
- Better UI

Problems

Complex coding

Old look (not modern)

3. JavaFX (Modern Framework)

Concept: Next-generation GUI framework.

Features

- Modern UI
- CSS styling
- Animation support
- Better performance

Comparison Table

Feature	AWT	Swing	JavaFX
Type	Heavyweight	Lightweight	Modern
UI Look	Basic	Better	Modern
Styling	No	Limited	CSS support
Animation	No	No	Yes
Performance	Low	Medium	High

Key Insight

JavaFX = Modern replacement of Swing

3. Features of JavaFX (Very Important)

1. Modern UI Design

Concept: JavaFX supports **modern, attractive user interfaces**.

Example

- Buttons with styling
- Responsive layout
- Professional look

Key Idea: Not old Java UI → modern application UI

2. CSS Styling Support

Concept: JavaFX allows styling using **CSS (like web development)**.

Example

```
btn.setStyle("-fx-background-color: blue; -fx-text-fill: white;");
```

Benefit

Separate design from logic

3. Animation Support

Concept: JavaFX supports **animations and transitions**.

Example Use

- Moving objects
- Fade effects
- Bounce animation

Example (Concept)

Ball moving left-right

Why Important

Benefit	Explanation
Interactive UI	Engaging
Better UX	Smooth experience

4. Scene Graph Architecture

Concept: JavaFX uses a **tree structure (hierarchy of nodes)**.

Example

Stage

└ Scene

└ Layout

└ Button

Benefit

Easy UI management

5. Rich UI Controls

Components

- Button
- TextField
- Label
- ComboBox

Benefit: Ready-to-use UI elements

6. Event Handling

Concept: JavaFX reacts to **user actions**.

Example:

```
btn.setOnAction(e -> System.out.println("Clicked"));
```

Key Idea: User action → event → response

7. Cross-Platform

- Runs on Windows, Linux, Mac
- Same code everywhere

8. Integration with FXML (Advanced)

- UI can be designed separately
- Cleaner code structure

4. Real-World Applications of JavaFX

Examples

Application	Use
Desktop apps	Calculator, editor
Management systems	Student, banking
Visualization tools	Charts, dashboards
Games	Simple animations

Example

Registration form:

Textbox + Button + Validation

Final Understanding

JavaFX:

- ✓ Modern GUI framework
- ✓ Replaces AWT and Swing
- ✓ Supports CSS and animation
- ✓ Uses Stage → Scene → Node

Unit-8: Basic Structure of JavaFX Application

Every JavaFX program follows a **fixed structure**.

Understanding this structure is the **foundation** for building any GUI application.

🖼️ Overall Structure (Big Picture)

Application → Stage → Scene → Node

1. Application Class (Entry Point)

Definition: Every JavaFX program must **extend the Application class**.

Why Needed

- It is the **starting point of JavaFX**
- It connects your program to JavaFX runtime

Syntax

```
import javafx.application.Application;
```

```
public class Test extends Application {  
}
```

Key Method

```
public static void main(String[] args) {  
    launch(args);  
}
```

What launch() Does

Starts JavaFX runtime → calls start() method

Key Understanding

Application class = Entry point of JavaFX app

2. start() Method (UI Initialization)

Definition: The start() method is where **UI is created and initialized**.

Syntax

```
public void start(Stage stage) {  
}
```

Important Points

Feature	Explanation
Called automatically	By JavaFX runtime
Receives Stage object	Main window
Used to build UI	Yes

Example

```
public void start(Stage stage) {  
    System.out.println("UI starts here");  
}
```

Key Insight

start() = where GUI is designed

3. Stage (Main Window)

Definition: A Stage represents the main window of a JavaFX application.

Real Understanding

Stage = Application Window

Example

```
stage.setTitle("My App");  
stage.show();
```

Important Methods

Method	Purpose
setTitle()	Set window title
setScene()	Attach scene
show()	Display window

Key Insight

Without stage.show() → window will not appear

4. Scene (Content Container)

Definition: A Scene holds all UI elements (nodes) inside the stage.

Real Understanding

Stage → window

Scene → content inside window

Syntax: Scene scene=new Scene(root,width,height);

Example

```
Scene scene=new Scene(new StackPane(),300,200);
stage.setScene(scene);
```

Key Points

Feature	Explanation
Contains UI elements	Yes
Attached to stage	Yes
Defines size	Yes

Key Insight

Scene = container of all UI elements

5. Node (UI Elements)

Definition: A Node is any visual element in JavaFX.

Examples of Nodes

Node Type	Example
Controls	Button, Label
Layouts	VBox, HBox
Shapes	Circle, Rectangle
Text	Text

Example

```
Button btn=new Button("Click Me");
```

Key Understanding

Everything in JavaFX is a Node

6. Complete Structure Example

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
```

```
public class Test extends Application {
    public void start(Stage stage) {
        Button btn=new Button("Click Me");
        StackPane root=new StackPane(btn);
        Scene scene=new Scene(root,300,200);
        stage.setTitle("JavaFX Example");
        stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

7. Execution Flow (Step-by-Step)

main() → launch()

↓
JavaFX runtime starts
↓
start(Stage stage) called
↓
Create nodes (Button, Layout)
↓
Create Scene
↓
Attach Scene to Stage
↓
stage.show() → display window

8. Visual Hierarchy (Very Important)

Stage
├─ Scene
│ └─ Layout (Pane)
│ └─ Nodes (Button, Text, etc.)

9. Real-World Example

Login Form

Stage → window

Scene → form layout

Nodes → TextField, Button, Label

Application Flow

User opens app → sees window → interacts with UI elements

10. Key Summary Table

Component	Role
Application	Entry point
start()	Initialize UI
Stage	Main window
Scene	Container
Node	UI element

Final Understanding

JavaFX Structure:

Application → start()
↓
Stage (window)
↓
Scene (container)
↓
Nodes (UI elements)

Goal:

- ✓ Build GUI step-by-step
- ✓ Organize UI properly
- ✓ Create interactive applications

Unit-8: JavaFX Architecture

JavaFX follows a **structured architecture** that organizes UI into a **hierarchical tree**. Understanding this is essential to design clean and scalable GUI applications.

1. Stage → Scene → Node Hierarchy (UI Structure)

Core Idea : Stage → Scene → Layout → Nodes

Explanation of Each Layer

Component	Role
Stage	Main window
Scene	Container inside window
Layout	Organizes UI
Node	Actual UI elements

Deep Understanding

1. Stage

- Top-level container
- Represents the **application window**

2. Scene

- Holds all UI content
- Attached to Stage

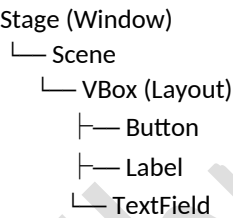
3. Layout Pane

- Organizes nodes
- Example: VBox, HBox

4. Node

- Smallest unit
- Example: Button, TextField

Example Structure



Code Example

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;

public class Test extends Application {
    public void start(Stage stage) {
        Button btn=new Button("Click");
        Label label=new Label("Name");
        TextField tf=new TextField();
        VBox root=new VBox(label,tf,btn);
        Scene scene=new Scene(root,300,200);
        stage.setScene(scene);
        stage.show();
    }
}
```

```
}  
public static void main(String[] args) {  
    launch(args);  
}  
}
```

Key Insight

UI is built layer by layer

2. Scene Graph (Tree Structure of UI)

Definition: A Scene Graph is a tree-like structure that represents all UI elements (nodes).

Core Idea

Scene Graph = Tree of Nodes

Structure

- Root node (top)
- Child nodes
- Parent-child relationship

Example

VBox (Root)

├─ Label

├─ TextField

└─ Button

Important Properties

Feature	Explanation
Hierarchical	Tree structure
Parent-child relation	Yes
Root node required	Yes
Dynamic updates	Possible

Why Scene Graph is Important

- Easy UI management
- Efficient rendering
- Automatic layout handling

Visual Understanding

Root Node

↓

Children Nodes

↓

Sub-children

Key Insight: Every UI element is part of Scene Graph

3. Layout vs Controls (UI Organization)

Layout (Container)

Definition: Layouts are containers used to arrange UI elements.

Examples

Layout	Use
VBox	Vertical arrangement

HBox	Horizontal arrangement
GridPane	Table structure
BorderPane	Regions

Example

```
VBox layout=new VBox();
layout.getChildren().add(new Button("OK"));
```

Key Idea: Layout manages position of nodes

Controls (UI Components)

Definition: Controls are **interactive UI elements used by users.**

Examples

Control	Purpose
Button	Click action
TextField	Input
Label	Display text
CheckBox	Selection

Example: Button btn=new Button("Submit");

Key Idea: Controls = user interaction elements

Layout vs Controls (Comparison)

Feature	Layout	Controls
Purpose	Arrange UI	Interact with user
Type	Container	Component
Example	VBox	Button
Role	Structure	Functionality

Combined Example

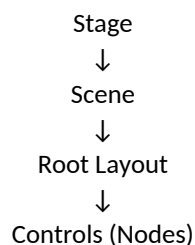
```
VBox root=new VBox();
Button btn=new Button("Click");
root.getChildren().add(btn);
```

Understanding

Layout → structure

Controls → content

4. Complete Architecture Flow



5. Real-World Example

Login Form

Stage → Window

Scene → Form

Layout → VBox/GridPane

Controls → TextField, Button

UI Representation

VBox

- └ Username Label
- └ TextField
- └ Password Field
- └ Login Button

6. Key Understanding

JavaFX Architecture:

- ✓ Stage → window
- ✓ Scene → container
- ✓ Scene Graph → tree structure
- ✓ Layout → organizes UI
- ✓ Controls → user interaction

Unit-8: Layout Panes

Layouts in JavaFX are **containers used to arrange UI elements (nodes)**.

They control **position, alignment, spacing, and structure** of the interface.

🧠 Core Idea

Layout Pane = Container that arranges UI elements

1. Pane (Basic Container – Free Positioning)

Concept: Pane is the **simplest layout** where elements are placed using **manual coordinates**.

Key Feature

- No automatic arrangement
- Developer sets position manually

Example

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.Pane;

public class Test extends Application {
    public void start(Stage stage) {
        Pane root=new Pane();
        Button btn=new Button("Click");
        btn.setLayoutX(100);
        btn.setLayoutY(50);
        root.getChildren().add(btn);
        Scene scene=new Scene(root,300,200);
        stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Use Case: Custom UI where exact positioning is needed

2. VBox (Vertical Layout)

Concept: Arranges elements **vertically (top → bottom)**.

Example

```
import javafx.scene.layout.VBox;
import javafx.scene.control.Button;
```

```
VBox vbox=new VBox();
vbox.getChildren().addAll(
    new Button("One"),
    new Button("Two"),
    new Button("Three")
);
```

Output Structure

One
Two
Three

Features

Feature	Explanation
Vertical alignment	Yes
Spacing support	Yes
Easy layout	Yes

Key Insight

VBox = Column layout

3. HBox (Horizontal Layout)

Concept :Arranges elements **horizontally (left → right)**.

Example

```
import javafx.scene.layout.HBox;
import javafx.scene.control.Button;
```

```
HBox hbox=new HBox();
hbox.getChildren().addAll(
    new Button("A"),
    new Button("B"),
    new Button("C")
);
```

Output Structure

A B C

Features

Feature	Explanation
Horizontal layout	Yes
Spacing control	Yes
Simple alignment	Yes

Key Insight

HBox = Row layout

4. GridPane (Table-like Layout)

Concept: Arranges elements in a **grid (rows and columns)**.

Syntax

```
grid.add(node, column, row);
```

Example

```
import javafx.scene.layout.GridPane;  
import javafx.scene.control.Label;  
import javafx.scene.control.TextField;
```

```
GridPane grid=new GridPane();  
grid.add(new Label("Name:"),0,0);  
grid.add(new TextField(),1,0);  
grid.add(new Label("Age:"),0,1);  
grid.add(new TextField(),1,1);
```

Output Structure

Name: [TextField]

Age: [TextField]

Features

Feature	Explanation
Rows & columns	Yes
Form design	Easy
Flexible layout	Yes

Key Insight

GridPane = Table layout (best for forms)

5. BorderPane (Top, Bottom, Left, Right)

Concept

Divides UI into **5 regions**:

Top

Left Center Right

Bottom

Example

```
import javafx.scene.layout.BorderPane;  
import javafx.scene.control.Button;
```

```
BorderPane bp=new BorderPane();  
bp.setTop(new Button("Top"));  
bp.setBottom(new Button("Bottom"));  
bp.setLeft(new Button("Left"));  
bp.setRight(new Button("Right"));  
bp.setCenter(new Button("Center"));
```

Structure

Top

Left Center Right

Bottom

Features

Region	Purpose
--------	---------

Top	Header
Bottom	Footer
Left	Menu
Right	Sidebar
Center	Main content

Key Insight

BorderPane = Layout for full application structure

6. StackPane (Overlapping Nodes)

Concept: Places elements **on top of each other (stacked)**.

Example

```
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
```

```
StackPane stack=new StackPane();
stack.getChildren().addAll(
    new Label("Background"),
    new Button("Click")
);
```

Output

Button overlaps Label

Features

Feature	Explanation
Overlapping nodes	Yes
Center alignment	Default
Used for layering	Yes

Key Insight

StackPane = Layered UI (top of each other)

7. Comparison of Layout Panes

Layout	Arrangement	Best Use
Pane	Free positioning	Custom design
VBox	Vertical	Forms, menus
HBox	Horizontal	Toolbars
GridPane	Rows/Columns	Forms
BorderPane	5 regions	Full UI layout
StackPane	Overlapping	Layers, backgrounds

8. Real-World Example

Login Form

GridPane → Form structure

VBox → Overall layout

HBox → Buttons alignment

Example Structure

VBox

├ Title

├ GridPane (Username, Password)

↳ HBox (Login, Cancel)

Final Understanding

Layout Panes:

- ✓ Pane → manual positioning
- ✓ VBox → vertical layout
- ✓ HBox → horizontal layout
- ✓ GridPane → table structure
- ✓ BorderPane → full UI structure
- ✓ StackPane → layered UI

Unit-8: UI Components (Controls)

Controls are **interactive UI elements** that users see and interact with.

They are all **Nodes** in JavaFX and are typically placed inside a **Layout Pane**.

🧠 Core Idea

Controls = UI elements that take input or show output

1. Label (Display Text)

Concept: Label is used to **display non-editable text**.

Example

```
import javafx.scene.control.Label;
```

```
Label label=new Label("Welcome to JavaFX");
```

Use Cases

- Titles
- Instructions
- Messages

Key Points

Feature	Explanation
Editable	No
Displays text	Yes
Lightweight	Yes

Insight

Label = Display only (no input)

2. Button (Click Action)

Concept: Button is used to **perform an action when clicked**.

Example

```
import javafx.scene.control.Button;
```

```
Button btn=new Button("Click Me");  
btn.setOnAction(e -> System.out.println("Button Clicked"));
```

Use Cases

- Submit form
- Start/Stop actions
- Navigation

Key Points

Feature	Explanation
Clickable	Yes
Event handling	Yes
Action trigger	Yes

Insight

Button = Action trigger

3. TextField (Single-line Input)

Concept: TextField allows user to **enter single-line text**.

Example

```
import javafx.scene.control.TextField;
```

```
TextField tf=new TextField();
tf.setPromptText("Enter your name");
```

Getting Input

```
String text=tf.getText();
```

Use Cases

- Username
- Search bar
- Input fields

Insight

TextField = Single-line input

4. TextArea (Multi-line Input)

Concept: TextArea allows **multi-line text input**.

Example

```
import javafx.scene.control.TextArea;
```

```
TextArea ta=new TextArea();
ta.setPromptText("Enter description");
```

Use Cases

- Comments
- Feedback
- Notes

Key Difference

TextField → single line

TextArea → multiple lines

5. CheckBox (Multiple Selection)

Concept: CheckBox allows **multiple options to be selected**.

Example

```
import javafx.scene.control.CheckBox;
```

```
CheckBox cb=new CheckBox("Java");
cb.setSelected(true);
```

Check Status

```
boolean selected=cb.isSelected();
```

Use Cases

- Preferences
- Multiple choices
- Settings

Insight

CheckBox = Multiple selection allowed

6. RadioButton (Single Choice)

Concept: RadioButton allows **only one option to be selected from a group**.

Example

```
import javafx.scene.control.RadioButton;  
import javafx.scene.control.ToggleGroup;
```

```
RadioButton r1=new RadioButton("Male");  
RadioButton r2=new RadioButton("Female");
```

```
ToggleGroup group=new ToggleGroup();  
r1.setToggleGroup(group);  
r2.setToggleGroup(group);
```

Key Concept

ToggleGroup ensures only one selection

Use Cases

- Gender selection
- Single option selection

Insight

RadioButton = Single selection

7. ComboBox (Dropdown List)

Concept: ComboBox provides a **dropdown list of options**.

Example

```
import javafx.scene.control.ComboBox;
```

```
ComboBox<String> cb=new ComboBox<>();  
cb.getItems().addAll("Java","Python","C++");
```

Get Selected Value

```
String value=cb.getValue();
```

Use Cases

- Country selection
- Category selection
- Options list

Insight

ComboBox = Dropdown selection

8. Combined Example (All Controls Together)

```
import javafx.application.Application;
```

```

import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.VBox;
import javafx.scene.control.*;

public class Test extends Application {
    public void start(Stage stage) {
        Label label=new Label("User Form");
        TextField tf=new TextField();
        TextArea ta=new TextArea();
        CheckBox cb=new CheckBox("Accept Terms");
        RadioButton r1=new RadioButton("Male");
        RadioButton r2=new RadioButton("Female");
        ToggleGroup group=new ToggleGroup();
        r1.setToggleGroup(group);
        r2.setToggleGroup(group);
        ComboBox<String> combo=new ComboBox<>();
        combo.getItems().addAll("Java","Python");
        Button btn=new Button("Submit");
        VBox root=new VBox(label,tf,ta,cb,r1,r2,combo,btn);
        Scene scene=new Scene(root,300,400);
        stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}

```

9. Comparison Table

Control	Purpose	Input Type
Label	Display text	None
Button	Perform action	Click
TextField	Input	Single-line
TextArea	Input	Multi-line
CheckBox	Selection	Multiple
RadioButton	Selection	Single
ComboBox	Selection	Dropdown

10. Real-World Example

Registration Form

Label → Title

TextField → Name

TextArea → Address

CheckBox → Terms

RadioButton → Gender

ComboBox → Course

Button → Submit

Final Understanding

UI Controls:

- ✓ Label → display text

- ✓ Button → trigger action
- ✓ TextField → single input
- ✓ TextArea → multi input
- ✓ CheckBox → multiple selection
- ✓ RadioButton → single selection
- ✓ ComboBox → dropdown

Unit-8: Text, Font, Color

This part focuses on **visual styling of UI** in JavaFX—how text looks, how fonts are applied, and how colors improve design.

Core Idea

Text + Font + Color → UI appearance & styling

1. Text Class (Display Styled Text)

Concept: Text is used to **display styled and customizable text**.

🔗 More powerful than Label for styling.

Example

```
import javafx.scene.text.Text;
```

```
Text text=new Text("Hello JavaFX");
```

Important Methods

Method	Purpose
setText()	Set text
setX(), setY()	Position
setFill()	Text color
setFont()	Apply font

Example with Styling

```
import javafx.scene.text.Text;
import javafx.scene.paint.Color;
```

```
Text text=new Text("Styled Text");
text.setFill(Color.BLUE);
```

Key Insight

Text = Rich styled text (more control than Label)

2. Font Class (Font Style & Size)

Concept: Font is used to **set font type, size, and style of text**.

Syntax

```
import javafx.scene.text.Font;
```

```
text.setFont(new Font("Arial",20));
```

Example

```
import javafx.scene.text.Text;
import javafx.scene.text.Font;
```

```
Text text=new Text("JavaFX Font");
text.setFont(new Font("Verdana",25));
```

Font Styles

Style	Description
Normal	Default
Bold	Thick text
Italic	Slanted text

Example with Style

```
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
```

```
text.setFont(Font.font("Arial",FontWeight.BOLD,20));
```

Key Insight

Font controls appearance of text

3. Color Class (Text & UI Color)

Concept: Color is used to **set color of text and UI elements.**

Example

```
import javafx.scene.paint.Color;
```

```
text.setFill(Color.RED);
```

Common Colors

Color	Usage
Color.RED	Error
Color.GREEN	Success
Color.BLUE	Information
Color.BLACK	Default

Custom Color

```
text.setFill(Color.rgb(0,128,255));
```

Applying Color to UI

```
Button btn=new Button("Click");
btn.setStyle("-fx-background-color: yellow;");
```

Key Insight

Color enhances UI visibility and design

4. Styling UI (Basic Design)

JavaFX supports **basic styling using methods and CSS-like syntax.**

Method 1: Inline Styling

```
btn.setStyle("-fx-background-color: blue; -fx-text-fill: white;");
```

Method 2: Using Font + Color Together

```
Text text=new Text("Hello");
text.setFont(new Font("Arial",20));
text.setFill(Color.GREEN);
```

Method 3: Layout Styling

```
VBox root=new VBox();
root.setStyle("-fx-padding: 20; -fx-spacing: 10;");
```

Common CSS Properties

Property	Use
-fx-background-color	Background
-fx-text-fill	Text color
-fx-font-size	Text size
-fx-padding	Space inside
-fx-spacing	Space between nodes

Example

```
Button btn=new Button("Styled Button");
btn.setStyle("-fx-background-color: green; -fx-text-fill: white; -fx-font-size: 14px;");
```

5. Combined Example (Text + Font + Color)

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.VBox;
import javafx.scene.text.Text;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.paint.Color;
```

```
public class Test extends Application {
    public void start(Stage stage) {
        Text text=new Text("JavaFX Styling");
        text.setFont(Font.font("Arial",FontWeight.BOLD,25));
        text.setFill(Color.BLUE);
        VBox root=new VBox(text);
        root.setStyle("-fx-padding: 20;");
        Scene scene=new Scene(root,300,200);
        stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

6. Real-World Example

Login Page UI

Title → Large bold text

Input labels → medium font

Buttons → colored background

Styling Flow

Text → display title

Font → make it bold and big

Color → highlight UI

CSS → improve design

7. Comparison: Text vs Label

Feature	Text	Label
Styling	Advanced	Basic
Position control	Yes	Limited

Use	Design-heavy text	Simple text
-----	-------------------	-------------

Final Understanding

Text → display styled text
 Font → control style and size
 Color → enhance UI appearance
 Styling → make UI attractive

Unit-8: Event Handling in JavaFX

Event handling makes the UI **interactive**.
 It defines **how the application responds to user actions** like clicks, typing, selection.

🧠 **Core Idea:** User action → Event → Handler → Response

1. Event Concept (User Interaction)

Definition: An event is an action performed by the user or system.

Examples of Events

Action	Event
Click button	ActionEvent
Type text	KeyEvent
Move mouse	MouseEvent

Real Understanding

User clicks → event generated → program responds

Example

Click "Submit" → show message

2. ActionEvent (Button Click)

Concept: ActionEvent is the most common event used in JavaFX.

🔗 Triggered when:

- Button is clicked
- Menu item selected
- Enter key pressed

Example

```
import javafx.scene.control.Button;
```

```
Button btn=new Button("Click");
btn.setOnAction(e -> System.out.println("Button clicked"));
```

Key Insight

ActionEvent = event for user actions

3. Event Handler (Handling Actions)

Definition: An event handler is code that executes when an event occurs.

Syntax

```
btn.setOnAction(handler);
```

Traditional Method

```
btn.setOnAction(new EventHandler<ActionEvent>() {
    public void handle(ActionEvent e) {
        System.out.println("Clicked");
    }
});
```

Understanding

Event occurs → handler executes → action performed

4. Lambda Expressions (Clean Event Code)

Concept: Lambda expressions provide a **short and clean way to write event handlers**.

Syntax

```
btn.setOnAction(e -> {  
    System.out.println("Clicked");  
});
```

Comparison

Approach	Code Length
Traditional	Long
Lambda	Short

Example

```
Button btn=new Button("Submit");  
btn.setOnAction(e -> System.out.println("Submitted"));
```

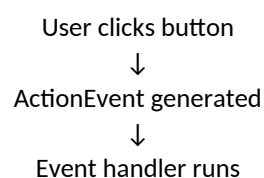
Key Insight

Lambda = cleaner and modern coding style

5. Complete Example (Event Handling)

```
import javafx.application.Application;  
import javafx.stage.Stage;  
import javafx.scene.Scene;  
import javafx.scene.layout.VBox;  
import javafx.scene.control.Button;  
import javafx.scene.control.Label;  
  
public class Test extends Application {  
    public void start(Stage stage) {  
        Label label=new Label("Click button");  
        Button btn=new Button("Click Me");  
        btn.setOnAction(e -> label.setText("Button Clicked"));  
        VBox root=new VBox(label,btn);  
        Scene scene=new Scene(root,300,200);  
        stage.setScene(scene);  
        stage.show();  
    }  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

Execution Flow





UI updates (Label text changes)

6. Types of Events (Basic Overview)

Event Type	Example
ActionEvent	Button click
MouseEvent	Mouse click/move
KeyEvent	Keyboard input

7. Real-World Examples

1. Login Button

Click → validate input → show message

2. Calculator

Button click → perform calculation

3. Form Submission

Submit → save data → show confirmation

8. Key Concepts Summary

Concept	Meaning
Event	User action
ActionEvent	Button click event
Event Handler	Handles event
Lambda	Short syntax

Final Understanding

Event Handling:

- ✓ Event → user action
- ✓ ActionEvent → button click
- ✓ Handler → response logic
- ✓ Lambda → clean code

Unit-8: JavaFX Properties & Animation

8. JavaFX Properties (Basic)

JavaFX introduces a powerful concept called **properties**, which enable **dynamic and automatic UI updates**.

1. Observable Properties (Dynamic UI Updates)

Concept: An **observable property** is a **variable whose value can be observed and tracked**.

🔄 When value changes → UI updates automatically

Core Idea

Property value changes → UI updates automatically

Example

```
import javafx.beans.property.SimpleStringProperty;
```

```
SimpleStringProperty name=new SimpleStringProperty("Java");  
name.set("JavaFX");  
System.out.println(name.get());
```

Why "Observable"?

Because:

- It can **notify changes**

- UI components can **listen to it**

Example with UI

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.beans.property.SimpleStringProperty;
```

```
public class Test extends Application {
    public void start(Stage stage) {
        SimpleStringProperty textProp=new SimpleStringProperty("Hello");
        Label label=new Label();
        label.textProperty().bind(textProp);
        textProp.set("Updated Text");
        VBox root=new VBox(label);
        Scene scene=new Scene(root,300,200);
        stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

What Happens

textProp changes → Label updates automatically

Key Insight

Observable property = data + automatic update mechanism

2. Binding Concept (Auto UI Updates)

Definition: Binding connects one property to another, so when one changes, the other updates automatically.

Types of Binding

Type	Description
One-way binding	One depends on another
Two-way binding	Both update each other

One-Way Binding Example

```
Label label=new Label();
TextField tf=new TextField();
label.textProperty().bind(tf.textProperty());
```

Behavior

User types in TextField → Label updates automatically

Two-Way Binding Example

```
tf1.textProperty().bindBidirectional(tf2.textProperty());
```

Behavior

Change tf1 → tf2 updates

Change tf2 → tf1 updates

Key Insight: Binding removes need for manual update code

3. Real-World Example (Properties + Binding)

Example: Live Form Preview

User enters name → label shows it instantly

Code Concept

```
label.textProperty().bind(tf.textProperty());
```

Benefit

- Real-time UI updates
- Clean code
- Less manual handling

9. Animation (Basic Idea)

Animation makes UI **dynamic and interactive**.

1. Timeline (Frame-Based Animation)

Concept: Timeline is used to create **animations using key frames (time-based changes)**.

Core Idea: Time → frame → property change → animation

Example (Simple Animation)

```
import javafx.animation.*;
import javafx.util.Duration;
import javafx.scene.shape.Circle;

Circle circle=new Circle(50);

Timeline timeline=new Timeline(
    new KeyFrame(Duration.seconds(1),
        new KeyValue(circle.translateXProperty(),200))
);
timeline.play();
```

Explanation

After 1 second → circle moves to X=200

Key Components

Component	Role
Timeline	Controls animation
KeyFrame	Time point
KeyValue	Property change

2. Transition Classes (Movement)

JavaFX provides **ready-made animation classes**.

Common Transitions

Transition	Purpose
TranslateTransition	Move object
FadeTransition	Fade in/out
ScaleTransition	Resize
RotateTransition	Rotate

Example: Movement (TranslateTransition)

```
import javafx.animation.TranslateTransition;
```

```
import javafx.util.Duration;
import javafx.scene.control.Button;

Button btn=new Button("Move");

TranslateTransition tt=new TranslateTransition();
tt.setNode(btn);
tt.setDuration(Duration.seconds(2));
tt.setByX(200);
tt.play();
```

Behavior

Button moves horizontally

Example: Fade Effect

```
import javafx.animation.FadeTransition;
```

```
FadeTransition ft=new FadeTransition(Duration.seconds(2),btn);
ft.setFromValue(1.0);
ft.setToValue(0.0);
ft.play();
```

Behavior

Button fades out

3. Timeline vs Transition

Feature	Timeline	Transition
Control	Detailed	Simple
Setup	Complex	Easy
Use	Custom animation	Quick animation

4. Real-World Examples

1. Loading Animation

Progress bar moving

2. Game UI

Character movement

3. Dashboard

Graphs animation

4. Notification UI

Fade in/out message

Final Understanding

JavaFX Properties:

- ✓ Observable → track changes
- ✓ Binding → automatic updates

Animation:

- ✓ Timeline → frame-based control
- ✓ Transition → simple movement

Unit-8: Real-World GUI Applications

This section connects JavaFX concepts to **actual applications** used in industry.
You will understand how layouts, controls, events, and styling combine to build real systems.

🧠 **Core Idea:** UI Components + Layout + Events + Styling → Real Application

1. Forms (Registration UI)

Concept: Forms are used to **collect user input** such as name, email, password, etc.

Structure of a Form

Title

Name → TextField

Email → TextField

Gender → RadioButton

Course → ComboBox

Terms → CheckBox

Submit → Button

Key Components Used

Component	Purpose
GridPane	Layout (form structure)
Label	Field name
TextField	Input
RadioButton	Single choice
ComboBox	Dropdown
Button	Submit action

Example Code (Registration Form)

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.*;

public class Test extends Application {
    public void start(Stage stage) {
        GridPane grid=new GridPane();

        Label name=new Label("Name:");
        TextField tf=new TextField();

        Label email=new Label("Email:");
        TextField tf2=new TextField();

        Button btn=new Button("Submit");

        btn.setOnAction(e -> System.out.println("Form Submitted"));

        grid.add(name,0,0);
        grid.add(tf,1,0);
        grid.add(email,0,1);
        grid.add(tf2,1,1);
        grid.add(btn,1,2);
    }
}
```

```

        Scene scene=new Scene(grid,300,200);
        stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}

```

Real Use

- Registration forms
- Login systems
- Feedback forms

Key Insight

Forms = Data input system

2. Dashboards (Data Display)

Concept: Dashboards are used to **display and monitor data visually**.

Examples

- Admin panel
- Sales dashboard
- Analytics screen

Structure

Top → Title

Left → Menu

Center → Data display

Right → Info panel

Key Components

Component	Purpose
BorderPane	Layout
Label	Display data
TableView	Tabular data
Charts	Graphs

Example Code (Simple Dashboard)

```

import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.BorderPane;
import javafx.scene.control.Label;

public class Test extends Application {
    public void start(Stage stage) {
        BorderPane root=new BorderPane();

        root.setTop(new Label("Dashboard"));
        root.setCenter(new Label("Data Display Area"));
        root.setBottom(new Label("Status"));

        Scene scene=new Scene(root,400,300);
        stage.setScene(scene);
    }
}

```

```

        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}

```

Features

- Real-time updates
- Data visualization
- Interactive controls

Key Insight

Dashboard = Data visualization + control panel

3. Interactive Applications (Games, Tools)

Concept: Interactive apps respond dynamically to **user actions and events**.

Examples

Application	Description
Calculator	Button-based input
Drawing tool	Mouse interaction
Game	Animation + events

Example: Simple Button Game

```

import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;

public class Test extends Application {
    public void start(Stage stage) {
        Button btn=new Button("Click Me");

        btn.setOnAction(e -> btn.setText("Clicked!"));

        StackPane root=new StackPane(btn);
        Scene scene=new Scene(root,300,200);
        stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}

```

Features Used

Feature	Role
Event handling	User interaction
Animation	Movement
Controls	UI elements

Key Insight

Interactive apps = Event-driven + dynamic UI

4. Combined Real-World Application

Example: Student Management System

Form → Enter student data

Dashboard → Display students

Event → Save button

File → Store data

Flow

User input → Form → Event handling → Data processing → Display result

5. Mapping Concepts to Applications

Concept	Used In
Layouts	Form, dashboard
Controls	Input & interaction
Events	User actions
Styling	UI design
Animation	Games

6. Industry-Level Understanding

Modern applications combine:

GUI + Data + Logic + Events

Examples

- Banking system → Forms + validation
- E-commerce → Dashboard + cart
- Software tools → Interactive UI

Final Understanding

Real-World Applications:

- ✓ Forms → data input
- ✓ Dashboards → data display
- ✓ Interactive apps → user interaction

Goal:

- ✓ Build real systems
- ✓ Combine all JavaFX concepts
- ✓ Create user-friendly applications